TP 1: Calidad de Diseño y Construcción

Materia: Calidad de Software / Legislación
Carrera: Tecnicatura Universitaria en Programación (UTN)
Alumno: Ignacio Figueroa

Parte A: Análisis de Calidad

1. Identificación de Code Smells

Revisando el código de TurnoManager, encontré varios code smells bastante claros

- **Obsesión por Tipos Primitivos:** Se usan puros String, int y boolean para manejar todo. El problema principal es que guarda los turnos pegando todo en una sola cadena de texto (nombrePaciente + "-" + edad + ...) en lugar de crear un objeto Turno o Paciente de verdad.
- **Código Duplicado:** La línea System.out.println("Paciente premium"); está repetida dos veces idéntica (para OSDE y para SWISS). Podría resolverse evaluando ambas obras sociales juntas.
- **Demasiados 'if' Anidados (Estructura en Flecha):** El método procesar mete un if adentro de otro if adentro de otro if. Se hace re difícil de leer y seguir el hilo del código.
- **Comparación Booleana Innecesaria:** Escribir if (urgente == true) es redundante. Con poner if (urgente) alcanza y sobra.
- **Nombres de Métodos Muy Generales:** Los nombres procesar() y mostrar() no te dicen mucho de qué hacen exactamente. Son demasiado genéricos.

2. Problemas de Mantenibilidad

- **Hace de Todo (Mucha Responsabilidad):** La clase TurnoManager valida los datos, decide qué categoría es la obra social, le da formato al texto, guarda los turnos y encima imprime en la consola. Si mañana queremos cambiar la forma de imprimir o cómo se valida, hay que tocar la misma clase.
- **Datos Hardcodeados:** Nombres como "OSDE", "SWISS" y "PUBLICA" están escritos a mano directo en el código. Si agregan una obra social nueva en el hospital, hay que entrar al código a meter más ifs.
- **Atado a la Consola:** Imprime todo con System.out.println. Si el día de mañana queremos usar este código para una app web o una pantalla con interfaz gráfica, no nos sirve así como está.

3. Ejemplos de Deuda Técnica

- **Guardar Turnos como un Texto Concatenado:** Guardar "Juan-30-OSDE-URGENTE" sirvió para sacarlo rápido en la pandemia, pero es un atajo feo. Si después queremos filtrar los turnos para ver quiénes son mayores de 60 años, habría que romper ese texto con un split("-"), lo que es un dolor de cabeza y propenso a errores.
- **Validación Incompleta y Silenciosa:** Si te pasan un nombre vacío o una edad negativa,

el código simplemente no hace nada y no te avisa. Ignorar estos errores por entregar rápido es acumular deuda técnica que después salta en producción.

4. Impacto en el Costo del Cambio

Cuando el código está lleno de atajos y mala calidad interna, **cualquier cambio tonto sale carísimo en tiempo y esfuerzo**. Meter una función nueva requiere tocar código viejo con miedo a romper algo que ya funcionaba, y lleva más tiempo entender qué hace el programa que programar la nueva mejora.

Parte B: Calidad Interna vs. Calidad Externa

1. ¿Qué es la Calidad Externa?

Es lo que ve y siente la persona que usa el sistema (el usuario final o los administrativos del hospital). Por ejemplo: que la app guarde bien los turnos, que sea rápida, que no se cuelgue y que la pantalla sea fácil de usar.

2. ¿Qué es la Calidad Interna?

Es cómo está hecho el código por dentro, algo que el usuario no ve pero los programadores sufrimos o disfrutamos. Incluye que el código sea prolijo, fácil de entender, que las variables tengan nombres claros y que esté dividido en partes lógicas (modulardad).

3. ¿Puede un sistema funcionar bien para el usuario y tener mal código por dentro?

Sí, totalmente. Este sistema del hospital es el ejemplo perfecto: para el administrativo los turnos se guardan bien (tiene buena calidad externa), pero por dentro el código es un desorden, está hardcodeado y es difícil de tocar (mala calidad interna).

4. ¿Qué riesgos trae mantener la Deuda Técnica a largo plazo?

- Que cada vez tardemos más tiempo en agregar cosas nuevas ("pagar los intereses").
- Que al arreglar una cosa rompamos otra sin querer.
- Que los programadores se frustren trabajando con un código feo y enredado.

Parte C: Código Refactorizado (Java)

Acá está el código mejorado. Mantiene exactamente el mismo comportamiento por fuera (imprime las mismas cosas en consola y guarda el turno con el mismo formato), pero por dentro está mucho más ordenado:

```
import java.util.ArrayList;
import java.util.List;

// Clase para representar un Turno (evitamos guardar puros Strings
```

```
concatenados)
class Turno {
    private String nombrePaciente;
    private int edad;
    private String obraSocial;
    private boolean urgente;

    public Turno(String nombrePaciente, int edad, String obraSocial,
boolean urgente) {
        this.nombrePaciente = nombrePaciente;
        this.edad = edad;
        this.obraSocial = obraSocial;
        this.urgente = urgente;
    }

    // Armamos el formato del texto acá dentro
    @Override
    public String toString() {
        String resultado = nombrePaciente + "-" + edad + "-" +
obraSocial;
        if (urgente) {
            resultado += "-URGENTE";
        }
        return resultado;
    }
}

public class TurnoManager {

    private List<Turno> turnos = new ArrayList<>();

    public void procesar(String nombrePaciente, int edad, String
obraSocial, boolean urgente) {
        // Validamos primero y si está mal salimos rápido (evita
achicar el código con mil 'if')
        if (!esDatosValidos(nombrePaciente, edad)) {
            return;
        }

        imprimirCategoriaPaciente(obraSocial);

        Turno nuevoTurno = new Turno(nombrePaciente, edad, obraSocial,
urgente);
        turnos.add(nuevoTurno);

        System.out.println("Turno agregado");
    }
```

```
    // Método auxiliar para validar datos de forma clara
    private boolean esDatosValidos(String nombre, int edad) {
        return nombre != null && !nombre.equals("") && edad > 0;
    }

    // Método auxiliar para la lógica de la obra social
    private void imprimirCategoriaPaciente(String obraSocial) {
        if (obraSocial == null) return;

        if (obraSocial.equals("OSDE") || obraSocial.equals("SWISS")) {
            System.out.println("Paciente premium");
        } else if (obraSocial.equals("PUBLICA")) {
            System.out.println("Paciente publico");
        }
    }

    public void mostrar() {
        // Usamos un for-each simple en vez del for con índice 'i'
        for (Turno turno : turnos) {
            System.out.println(turno);
        }
    }
}
```

Explicación Sencilla de las Mejoras

1. **Creamos la clase Turno:** En lugar de guardar una lista de textos pegados con guiones, creamos una clase con sus atributos (nombre, edad, etc.). Así el código queda mucho más orientado a objetos.
2. **Juntamos el código duplicado:** En imprimirCategoriaPaciente juntamos OSDE y SWISS usando un || (OR), así no repetimos dos veces la misma impresión de "Paciente premium".
3. **Sacamos los if anidados:** Usamos validaciones tempranas al principio del método. Si los datos están mal, el programa no sigue bajando y evitamos la estructura en forma de flecha.
4. **Arreglamos detalles feos:** Cambiamos el if (urgente == true) por un simple if (urgente), y usamos un bucle for-each normal que es más fácil de leer.